Doctaculous

Pure-Go document toolkit
HTML rendering specimen

The Rendering Feature Tour

A single document that exercises every implemented slice of the HTML / CSS / image pipeline — typography, the box model, floats, flexbox, CSS grid, tables, positioning, overflow, stacking, and decoded images — fetched over HTTP and paginated onto Letter pages.

01 / TYPE

Typography & Inline Flow

Block stacking, inline wrapping, three font families, and inline atoms sharing one baseline.

Headings cascade from the UA sheet

Body copy is set in `TeX Gyre Termes`, a serif. Headings switch to `TeX Gyre Heros`, a grotesque. This paragraph carries enough text to wrap across several lines at the page measure, so greedy line-breaking, leading, and left alignment are all visible. Short inline runs such as `display:inline-block` sit on the same line as the surrounding prose.

An inline-block atom `BADGE` shrinks to fit its content and rests on the text baseline — the words before and after it stay on the same line and the same baseline.

> “Accept interfaces, return concrete types.” A pull-quote drawn with a gold left rule and the sans family.

Alignment

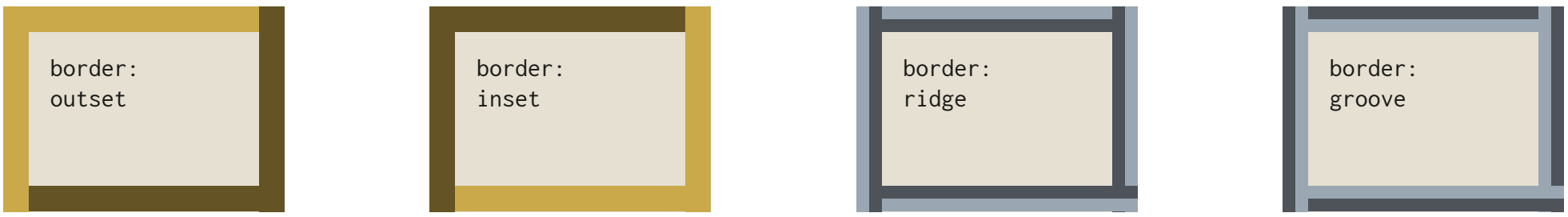
This line is centered.

This line is right-aligned.

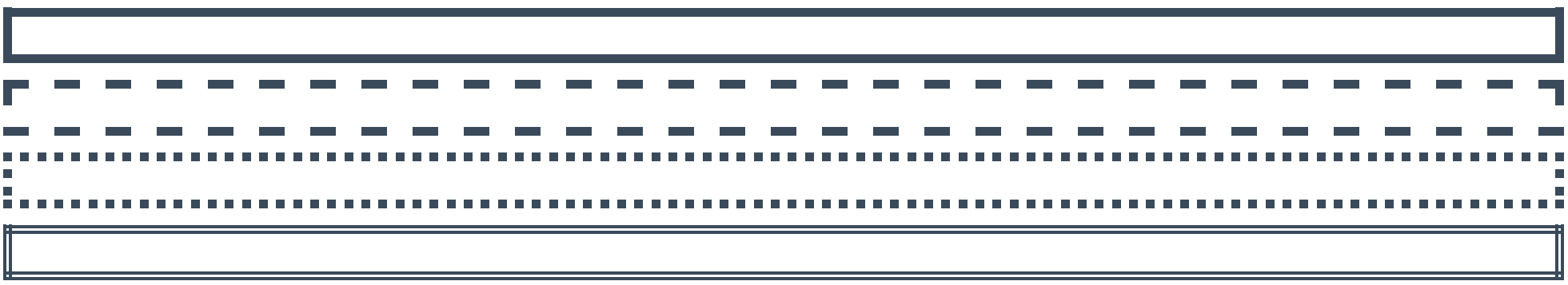
02 / BOX

The Box Model & Borders

Padding, margins, backgrounds, and every border style the engine renders — including the four 3D bevels.



Four `border-style` bevels. Outset reads raised, inset sunken; ridge and groove are their two-facet cousins.



Line styles: `solid`, `dashed`, `dotted`, `double`.

03 / FLOAT

Floats & Clear

A floated figure with text wrapping beside it, then a cleared block returning to full width.

img/photo.jpg · JPEG

This paragraph wraps its lines along the right edge of the left-floated figure. The decoded JPEG to the left is a continuous-tone image, so it exercises the baseline DCT decode path rather than a flat fill. As the text continues past the bottom of the figure box, the lines reclaim the full measure of the column and run edge to edge again, exactly as float behaviour requires. The figure keeps its margin gap so the text never touches the frame.

A `clear:left` block drops below the float and spans the full width beneath it.

04 / FLEX

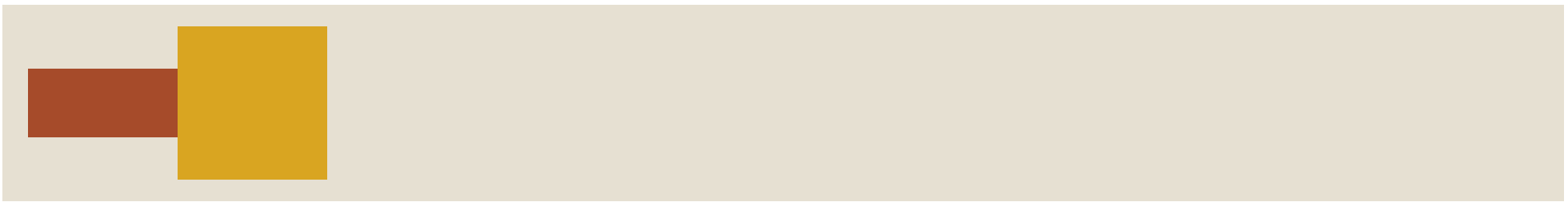
Flexbox

Proportional growth, space distribution, cross-axis centering, and multi-line wrapping with `align-content`.

`flex-grow` in 1 : 2 : 3 proportion

`justify-content: space-between`

`align-items: center`



A short and a tall item, both centered in the strip.

`flex-wrap: wrap` with `gap`

Items too wide to share one line break onto the next; the `gap` applies both between items and between the lines.

`align-content: space-between` on a taller container

With more cross space than the lines need, `align-content` distributes the surplus between them — here pinning the first and last lines to the container's edges. The default is `stretch`, which instead grows each line to absorb it.

05 / GRID

CSS Grid

Explicit tracks, named template areas, fractional units, spans, and auto-placement.

Named areas & `fr` tracks

A header spanning both columns over a 2fr main / 1fr side row.

Auto-placement with a spanning tile

The gold tile spans two columns; the rest auto-flow into the grid.

06 / TABLE

Tables

Collapsed borders, a caption, a spanning header, striped rows, and a total rule — then separate borders with column / row background layers.

Quarterly Ledger

Quarter	Region	Units	Revenue
Q1	North	1,204	$48,160
Q2	North	1,560	$62,400
Q3	South	1,028	$41,120
Total			**$151,680**

a1	b1	c1	d1
a2	b2	c2	d2
a3	b3	c3	d3

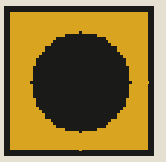
Separate borders: column 2 carries a tint, alternating rows a stripe; where they cross, the row layer wins.

07 / LAYER

Positioning, Overflow & Stacking

Absolute pinning, relative paint-time offset, overflow clipping, and z-index order.

This stage is `position:relative`. The gold badge is `position:absolute`, pinned to its top-right corner and painted above this text. The GIF inside it decodes through the indexed palette path.

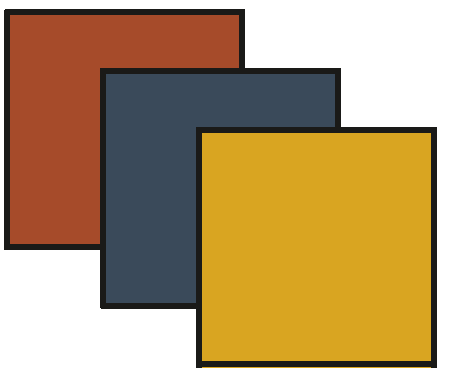


The rust tag is `position:relative`, nudged down and right from its in-flow slot at paint time.

`overflow: hidden`

A 320×320 child clipped to the padding box.

`z-index` order



Higher z paints on top: gold over slate over rust.

08 / IMAGE

Decoded Images & `object-fit`

One source square fitted three ways into wide frames.

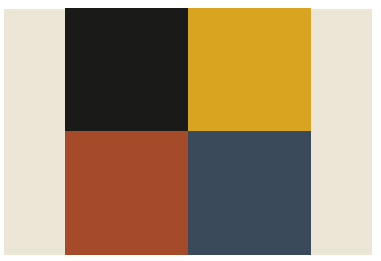
object-fit: contain

object-fit: cover

none + position

The four-quadrant PNG makes orientation unambiguous: contain letterboxes the whole square, cover fills and crops, and none pins the unscaled image to the bottom-right.

HEIC (pureGo HEVC decode)



PNG reference

The same quad image as an Apple-encoded HEIC decoded by the in-tree HEVC decoder, beside its lossless PNG twin — identical layout, near-identical pixels (HEIC is lossy 4:2:0).

09 / FORMS

Form Controls

Static, non-interactive native widgets — text fields, buttons, checkboxes, radios, a textarea, and a select — sized and painted with classic chrome.

Full name Ada Lovelace

Email you@example.com

Password •••••••

Notes Notes go here.

Plan Pro

Preferences

☑ Subscribe to updates

☐ Remember this device

◉ Contact by email

○ Contact by phone

Save changes Submit

10 / WHITE-SPACE

The white-space Property

How runs of spaces, tabs, and newlines are collapsed or preserved, and whether lines wrap.

pre — preserve everything, no wrap (tab-aligned)

```
name      role        team
ada       founder     core
grace     compiler    tools
margaret  navigation  apollo
```

Spaces, the literal newlines, and tab stops are preserved; long lines do not wrap (they overflow / are clipped by the box).

pre-wrap — preserve, but wrap

```
This preformatted block keeps    its    runs of spaces and its
explicit line breaks, but unlike pre it still wraps a line that is too long to fit within the width of its
containing box.
```

pre-line — collapse spaces, keep newlines

```
Spaces collapse to one,
but each
newline is kept.
```

nowrap — collapse, never wrap

```
This single line never wraps no matter how nar
```

The clip box (overflow:hidden) cuts the overflowing line at its edge.

11 / LISTS

Lists & Counters

Bullets and numbers, nested marker rotation, the numbering styles, and CSS counters.

Unordered (nested)

- Disc at the top level
- Another item
  - Circle one level in
    - Square two levels in
- Back to disc

Ordered styles

1. Decimal one
2. Decimal two

a. Lower-alpha
b. Lower-alpha

I. Roman
II. Roman
III. Roman
IV. Roman

Nested numbering with `counters()`

1. First section
   1. Subsection
   2. Subsection
2. Second section

The inner list's `counter-reset` opens a new scope; `counters(list-item, ".")` would join them as 1.1, 1.2 — here the default per-list numbering restarts each level.

12 / BACKGROUNDS

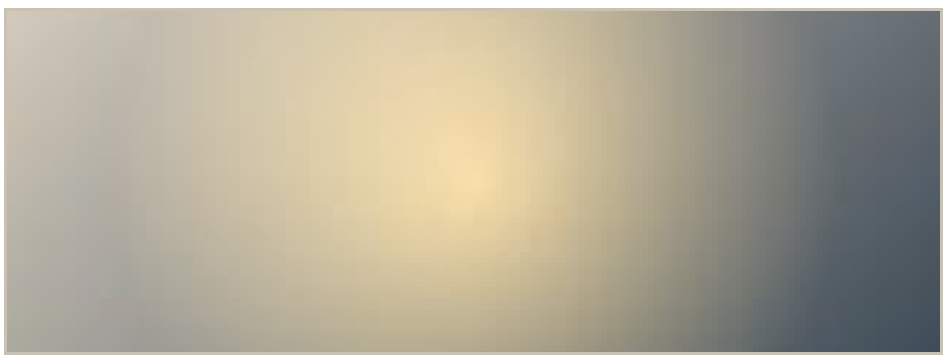
Background Images

A tiled texture, a cover hero, and a positioned badge — `background-image` with repeat, size and position.

Tiled texture (`repeat`)

A small seamless tile repeats to fill the panel. The text sits on top of the background, which paints behind the content and inside the padding box.

`cover`, centered



The photo scales to cover the box, cropping the overflow.

Positioned badge

A single icon, `no-repeat`, pinned bottom-right over a tint.

13 / LINKS

Links

The link pseudo-classes: a UA default, an author `a:link` color, an underline opt-out, and an inert `:visited` rule.

Body copy with a default hyperlink — blue and underlined, the user-agent style for an unvisited link.

A call-to-action link recolored with `a:link { color }`, keeping its underline.

A borderless link with `text-decoration: none` — colored but not underlined.

A `:visited` rule is parsed but never matches in a static render (no browsing history), so this link stays the unvisited style — and a bare anchor with no `href`, like this, is not a link at all.

14 / LEGACY ATTRIBUTES

Presentational Attributes

Pre-CSS styling via HTML attributes — `bgcolor`, `align`, `cellpadding`, `border`, `<font>` — mapped to CSS as below-cascade hints (real CSS still wins).

The kind of table the early web was built from: colored rows, padded bordered cells, and aligned columns, all set with attributes rather than a stylesheet.

Component	Language	Lines
Parser	Go	4,210
Layout engine	Go	11,930
Rasterizer	Go	6,740

An obsolete `<font>` still works: large red text, and an `<ol type="I" start="3">` numbers with the attribute:

<ol type="I" start="3">
Third in Roman
Fourth in Roman

15 / DIRECTION

RTL & Bidirectional Layout

`direction: rtl` runs the inline axis right-to-left. Tables mirror their column order, flex rows reverse their main axis, grids mirror their tracks, and the direction-relative `start`/`end` keywords resolve against it.

The table below sets `dir="rtl"`. Column one is the right-most, and the border-collapse grid lines mirror with it:

Third	Second	First
Leftmost cell	Middle cell	Rightmost cell

A flex row under `dir="rtl"` packs from the right. The items are authored one-two-three in source order, so item one takes the right-most slot:

A grid mirrors its tracks the same way — the `1fr 2fr` ratio is preserved, but the single-width track is now on the right:

Finally, `text-align` takes the logical keywords. Both paragraphs below use `text-align: end`; only their `direction` differs, so end resolves to opposite edges:

LTR — `end` resolves to the right edge

RTL — `end` resolves to the left edge

15 / DIRECTION

Real script

Everything above mirrors boxes. Text inside a line is reordered separately, per UAX#9: shaping and line-breaking run in logical order, and each line is reordered to visual order once the break is chosen. The Hebrew below is authored left-to-right in the source and reads right-to-left on the page:

שלום עולם

A right-to-left phrase inside an otherwise left-to-right paragraph reorders in place, and the Latin around it does not move — the bracket pairs mirror too (rule L4):

The greeting (שלום) sits inline.

Arabic additionally needs contextual shaping: a letter takes a different form depending on whether it joins to its neighbours, and some pairs fuse into one glyph. These are shaped through the font’s OpenType tables, so the letters connect rather than standing as isolated forms:

مرحبا بالعالم

16 / CROP

Exact-size Cropping

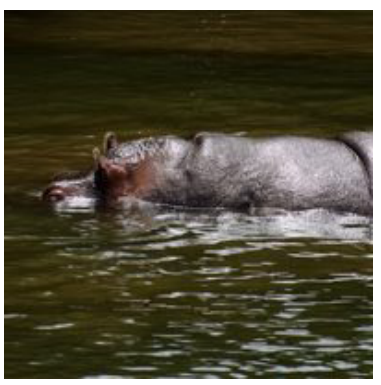
Filling an exact pixel box, rather than fitting within one.

`--max-width`/`--max-height` fit a render inside a box with aspect preserved, so a 3:2 source can never fill a square target — it comes back 300×200, not 200×200. `ImageOptions.Crop` fills the target instead, discarding what falls outside. The source below is 480×322:

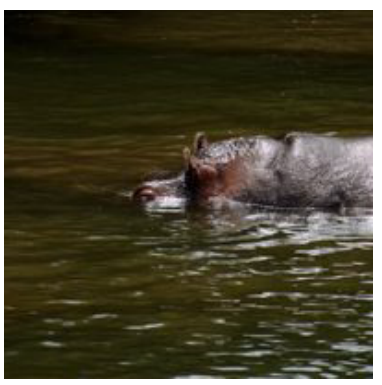
img/crop-source.jpg · 480×322 source

The same square target, placed three ways. Saliency scores candidate windows on edge energy, saturation, skin likelihood and a centre prior — no model, no training data — and picks the highest; the gravities place the window deterministically:

saliency



center



west

The subject sits left of centre, so the saliency window shifts toward it — `(20,0)-(342,322)` against the centre crop's `(79,0)-(401,322)`. `EncodeImageRect` returns that rectangle, so a caller can record where a smart crop landed and replay it later with `StrategyRect`.

17 / LANDSCAPE

Landscape Reflow

This section selects a wider `@page landscape` via `page: landscape`; its content reflows to the wider measure of a landscape US-Letter page (1056×816px) instead of the portrait column.

Because the named page is wider, the flex row below stretches edge to edge across the full landscape measure, and the table beneath it reflows to the same wide width — both visibly wider than any portrait page in this specimen. The running chrome (the head at top-left, the section title at top-right, and the page counters along the bottom) carries over onto the landscape page band unchanged.

Parser & xref

Content interpreter

Raster backend

Reflow engine

CSS cascade

Pipeline stage	Package	Input	Output	Notes
Parse	pkg/pdf	bytes	Document	xref tables & streams, object streams
Interpret	pkg/pdf/content	content stream	paint ops	paths, text, images, shadings
Reflow	pkg/layout/css	cssbox tree	fragment tree	block / inline / flex / grid / table
Raster	pkg/render/raster	paint ops	bitmap	pure-Go, dependency-light

---

Doctaculous · pure-Go, MIT-licensed · rendered from HTTP-fetched HTML, CSS, fonts and images, paginated onto US-Letter pages. Set in TeX Gyre Heros & Termes with Inconsolata; wordmark in Pacifico.